

Aula 16 - Estruturas de Controle: Condicionais e Laços de Repetição

Descrição: Aprofundamento nas estruturas de controle de fluxo lógico do JavaScript. Condicionais complexas utilizando switch para roteamento de decisões. Automatização de tarefas repetitivas através da implementação de laços de repetição for e while. Aplicação prática em algoritmos de varredura e processamento de dados para o Front-end.

Cenário Real: Pensem na nossa Situação de Aprendizagem (SA): em breve, teremos que exibir a lista de usuários cadastrados no nosso Dashboard, consumindo dados do banco. Imaginem ter que escrever manualmente a tag `<tr>` no HTML para cada um dos 5.000 clientes da empresa. Isso é humanamente impossível e tecnicamente inviável. Hoje, criaremos o "cérebro" da nossa página, dando a ela a habilidade de analisar milhares de dados, tomar decisões instantâneas e trabalhar por nós sem que precisemos copiar e colar código repetido.

Pergunta: Como fazemos o computador trabalhar por nós, processando regras complexas e repetindo uma tarefa milhares de vezes em milissegundos, consumindo pouca banda e economizando os recursos do nosso servidor?

Tomada de Decisão com Condicionais (if/else e Ternário)

A. Conceito:

A instrução `if` é a estrutura **fundamental** que permite ao JavaScript tomar decisões. Ela avalia uma expressão **entre parênteses** e, se o resultado for considerado **verdadeiro** (*truthy*), executa o bloco de código associado. No JavaScript, a avaliação converte automaticamente valores: itens como `undefined`, `null`, `0` e `""` (string vazia) são tratados como **falsos** (*falsy*), enquanto qualquer outra coisa (incluindo arrays) é verdadeira. Complementando isso, a cláusula `else` define uma **alternativa** a ser executada quando a expressão **falha**, permitindo criar um desvio claro do fluxo do sistema.

Quando precisamos lidar com múltiplas **ramificações** lógicas simples, utilizamos o padrão `else if`, **encadeando** validações sem prejudicar a leitura do código. Além disso, a linguagem oferece o operador condicional **ternário** (`?:`), que atua como um **atalho** sintático extremamente elegante para a estrutura `if/else` tradicional quando nosso único **objetivo** é **retornar** ou **atribuir** um **valor**. Esta abordagem diminui o número de linhas no arquivo, facilitando a manutenção e agilizando a nossa leitura, algo essencial na dinâmica do trabalho em equipe.

B. Visualizando:

Imaginem um fluxograma de uma portaria eletrônica. Há um bloco central de decisão em formato de losango: "O usuário enviou a senha?". Se a seta apontar para **SIM** (*truthy*), a cancela abre (bloco de código do `if`). Se apontar para **NÃO** (*falsy*), o sistema emite um alerta (bloco de código do `else`). O operador ternário seria a versão "**Via Expressa**" desse pedágio. {ver img01.png}

C. Exemplo de Código:

JavaScript

// Validando dados de um usuário na interface (truthy/falsy)

`let` nomeUsuario = `""`; // Falsy por ser string vazia

```
if (nomeUsuario) {  
  console.log("Bem-vindo, " + nomeUsuario);  
} else {  
  console.log("Erro: Preencha seu nome de usuário.");  
}
```

```
// Operador Ternário para definir status de um cardápio
let estoque = 5;
let statusProduto = (estoque > 0) ? "Disponível" : "Esgotado";
console.log("Status do Item: " + statusProduto);
```

D. Explicação linha por linha:

- `let nomeUsuario = ""`; Declaramos uma variável e a inicializamos com uma string vazia (considerada *false* em JS).
- `if (nomeUsuario) {`: O JavaScript tenta converter a string em booleano. Como é vazia, resulta em falso e o código pula o bloco.
- `else {`: Captura a falha na condição anterior.
- `console.log(...)`: Exibe a mensagem de erro alertando o preenchimento.
- `let statusProduto = ...`: Inicia a declaração de status aplicando o ternário.
- `(estoque > 0)`: É a nossa condição de teste.
- `? "Disponível"`: Valor atribuído caso o teste acima seja verdadeiro.
- `: "Esgotado"`: Valor atribuído caso o teste seja falso.

E. Exercícios Rápidos:

1. Crie um `if` que verifique se a variável `idade` é maior ou igual a 18 e imprima "Maior de idade".
2. Altere o exercício 1 adicionando um `else` para imprimir "Menor de idade".
3. Use o `else if` para imprimir "Idoso" se a idade for maior ou igual a 60.
4. Teste a validação *truthy*: passe o número 0 em um `if`. O que acontece?
5. Reescreva o exercício 1 e 2 em uma única linha utilizando o operador ternário.

Roteamento de Múltiplos Caminhos (switch)

A. Conceito:

A instrução `switch` é utilizada especificamente quando todas as nossas ramificações lógicas **dependem** da avaliação de uma **única** expressão contra vários valores diferentes. Em vez de criar um bloco gigante e confuso cheio de `else if` (que torna a auditoria de código difícil), o `switch` calcula o valor da expressão apenas uma vez e procura um rótulo `case` correspondente. Uma característica técnica essencial é que a comparação utiliza **obrigatoriamente** o operador de identidade estrita (`===`), ou seja, não há conversão automática de tipo durante a validação.

Dentro da estrutura de roteamento do `switch`, a **palavra-chave** `break` desempenha um papel crítico para a estabilidade do sistema. As cláusulas `case` especificam apenas o ponto de partida da execução. Se nós **esquecermos** o `break`, a execução continuará rodando ("**vazará**") para os casos seguintes, independentemente de coincidirem ou não. Para lidar com valores inesperados, utilizamos a cláusula `default`, garantindo que sempre haverá uma saída segura para o fluxo.

B. Visualizando:

Pensem num painel telefônico de uma central de atendimento (PABX). A ligação entra (expressão `switch`) e o sistema procura exatamente o ramal digitado (`case 1`, `case 2`). Assim que o telefone do setor toca e é atendido, a conexão precisa ser cortada com a central (`break`), senão a ligação vai tocar nos outros ramais abaixo sequencialmente, virando uma confusão. O `default` é a secretária eletrônica geral para quem digita um ramal inexistente. {ver img02.png}

C. Exemplo de Código:

JavaScript

// Roteamento de acessos no nosso futuro Dashboard

let nivelAcesso = 2; // 1: User, 2: Editor, 3: Admin

```
switch (nivelAcesso) {
  case 1:
    console.log("Tela: Visualização de Dados");
    break;
  case 2:
    console.log("Tela: Edição de Clientes Liberada");
    break;
  case 3:
    console.log("Tela: Painel de Controle Total");
    break;
  default:
    console.log("Erro: Nível de acesso desconhecido. Refaça o login.");
}
```

D. Explicação linha por linha:

- `let nivelAcesso = 2;`: Simulamos o retorno de um tipo de usuário vindo do banco de dados.
- `switch (nivelAcesso) {`: Inicia o painel, avaliando qual é o valor numérico exato da variável.
- `case 1:`: Pergunta "`nivelAcesso === 1?`". Não é, então ignora.
- `case 2:`: Pergunta "`nivelAcesso === 2?`". É verdade! O fluxo entra aqui.
- `console.log(...)`: Imprime que a edição foi liberada.
- `break;`: **Fundamental**. Interrompe o `switch` para impedir que o código caia no `case 3` por acidente.
- `default:`: É o "else" do switch. O que fazer se o valor for 4, 5, "dois", etc.
- `console.log(...)`: Mensagem de segurança por fallback.

E. Exercícios Rápidos:

1. Crie um `switch` que valide a variável `diaDaSemana` (1 a 7) e imprima o nome do dia.
2. Remova intencionalmente o `break` de um `case` do exercício 1 e rode o código. O que aconteceu?
3. O que acontece se a variável `diaDaSemana` receber a string "1" no lugar do número 1?
4. Adicione um `default` para informar "Dia inválido" se passar o número 8.
5. Pesquise rapidamente na IA: É possível colocar mais de um `case` executando a mesma linha de código (agrupamento)? Teste isso no seu VS Code.

Automatização com Laços de Repetição (for e while)

A. Conceito:

Os **laços de repetição** (Loops) são o **motor** de trabalho da **automação** no JavaScript, sendo capazes de **repetir** um bloco de código **incontáveis** vezes enquanto uma determinada condição for avaliada como **verdadeira**. O laço `for` é a ferramenta perfeita para cenários onde já **conhecemos** previamente o **limite** do processamento (ex: o tamanho de um array de clientes), pois consolida a criação da variável de controle, a condição e o incremento na mesma linha. Já a estrutura `while` é ideal quando a **repetição** depende de uma regra de negócio que **não tem um fim previsível**, como esperar um processo de servidor terminar.

Para mantermos a eficiência da aplicação e a ética profissional (entregando performance), devemos ser rigorosos com a condição de parada. O mau uso de laços gera "loops infinitos" que travam o navegador do cliente. Para refinar esse controle de fluxo, utilizamos `break` para sair totalmente do laço, e

`continue` (com ou sem Rótulos/Labels) para pular uma iteração específica que não queremos processar, poupando processamento e mantendo as diretrizes de código limpo.

B. Visualizando:

Pense em uma esteira de fábrica de sapatos. O laço `for` é o braço robótico que pinta os sapatos. A configuração dele diz: "comece na caixa 0 (`let i = 0`), continue operando se houver menos de 100 caixas (`i < 100`), após pintar, passe para a próxima (`i++`)". Se passar pela esteira um sapato com defeito, o robô recebe um aviso de `continue`: ele não pinta aquele sapato defeituoso, mas volta para a posição inicial para esperar o próximo imediatamente. {ver img03.png}

C. Exemplo de Código:

JavaScript

// Varrendo 5 registros fictícios para construir a lista do Front-end

```
let clientes = ["Ana", "Bruno", "Carlos", "Diana", "Eduardo"];
```

```
for (let i = 0; i < clientes.length; i++) {  
  // Regra de negócio: pular clientes cujo nome inicia com 'C'  
  if (clientes[i].startsWith("C")) {  
    console.log(`Puxa, o(a) ${clientes[i]} está com cadastro bloqueado. Pulando...`);  
    continue;  
  }  
  
  // Processamento pesado (simulação)  
  console.log(`Montando <tr> para: ${clientes[i]}`);  
}
```

D. Explicação linha por linha:

- `let clientes = [...]`: Array contendo nossa lista.
- `for (let i = 0;`: Passo 1 - Inicialização. Começamos a contar do índice 0 (primeiro item).
- `i < clientes.length;`: Passo 2 - Condição. Enquanto `i` for menor que 5 (tamanho do array), continue rodando.
- `i++`: Passo 3 - Incremento. Ao final de cada ciclo, aumente o `i` em 1.
- `if (clientes[i].startsWith("C"))`: Valida se a string na posição `i` começa com "C".
- `continue;`: Se começar com C, aborta a iteração atual (pulou o Carlos) e volta ao topo do `for` somando mais 1 no `i`.
- `console.log(...)`: Só vai imprimir as pessoas que não ativaram o `continue`.

E. Exercícios Rápidos:

1. Crie um `for` que conte de 1 a 10 imprimindo os números.
2. Modifique o exercício 1 para contar regressivamente de 10 até 1.
3. Crie um `while` que duplique o valor de uma variável `saldo = 10` até que passe de 100.
4. Utilize `break` para parar um loop `for` de 1 a 20 assim que ele bater no número 13.
5. Crie um array de produtos e use um laço para imprimir o nome de cada um deles.

FAQ - Dúvidas Reais

1. "Por que no `switch` precisamos de três iguais (`===`) na cabeça da gente e o `if` aceita dois iguais (`==`)?"

- **Resposta:** Porque o `switch` foi projetado para verificação de Identidade Estrita, evitando falsos positivos que causam bugs difíceis de rastrear. O uso de `==` (com coerção de tipos) no mercado já é desencorajado por boas práticas (Clean Code), então o JavaScript nos força a ser precisos no `switch`.

2. "Se eu posso fazer tudo com `while`, por que o `for` existe?"

- **Resposta:** Organização. O `for` agrupa inicialização, verificação e incremento em uma única linha. Fazer isso no `while` exige escrever mais linhas e aumenta a chance de você esquecer de somar o contador, travando a aplicação com um loop infinito.

3. "Eu posso colocar um `if` dentro de um `for`?"

- **Resposta:** Sempre! Isso é o que chamamos de aninhamento. Você varre os dados com o laço de repetição e, para cada item, usa condicionais para aplicar regras de negócios específicas (como no nosso exemplo do "Carlos").

4. "Usei o ChatGPT para achar um erro e ele sugeriu a função `eval()`, devo usar?"

- **Resposta:** Não! O uso de `eval()` é desencorajado por impedir otimizações do motor JavaScript e introduzir vulnerabilidades de segurança. O bom profissional de TI avalia o código sugerido pelas IAs, garantindo a robustez do software, em vez de aplicar atalhos que comprometem a segurança (Soft Skill: Pensamento Crítico).

5. "Como o operador ternário ajuda na performance de Front-end?"

- **Resposta:** Por sua característica lógica de "curto-circuito". Apenas um dos dois caminhos possíveis (antes ou depois dos dois pontos) é de fato processado pelo interpretador, o que poupa mili-segundos essenciais ao lidar com a renderização visual do DOM na web móvel.

Exercícios de Fixação

1. Desafio: Radar de Velocidade

Descrição: Crie um sistema em JS que defina uma variável `velocidade`. Se a velocidade for maior que 80, o console deve alertar "Multado!". Se não, "Dentro do limite".

Entrega: Script simples usando `if/else`.

2. Desafio: Contagem de Estoque

Descrição: Faça um loop `while` que tenha um total de 5 caixas. O laço deve decrementar uma a uma até que zere, mostrando "Faltam X caixas".

Entrega: Script rodando sem gerar loop infinito.

3. Desafio: Validador de Tipos

Descrição: Defina uma variável com qualquer valor (número, string, array, nulo). Crie um `switch` baseado no `typeof` dessa variável que informe ao usuário se o tipo é "Texto", "Numérico", "Objeto" ou "Desconhecido".

Entrega: Código funcional cobrindo no mínimo 3 retornos no `switch` e um `default`.

4. Desafio: Caixa Eletrônico (Ternário)

Descrição: Crie variáveis de `saldo` e `saqueDesejado`. Use um operador ternário que valide se há fundos. Retorne uma string "Saque Autorizado" ou "Saldo Insuficiente".

Entrega: Funcionalidade em 1 linha de código de validação, armazenada na variável de resposta.

5. Desafio: Pular os Pares

Descrição: Crie um laço `for` de 1 a 20. Use a palavra-chave `continue` associada a um `if` para ignorar todos os números pares e imprimir apenas os números ímpares no console.

Entrega: Código contendo o loop, o operador de módulo (%) e o uso correto do `continue`.

6. Desafio: O Menu do Dashboard

Descrição: Crie uma variável que recebe um pedido do usuário: "Relatorios", "Clientes", "Configuracoes" ou outro valor. Use um `switch` que, ao entrar na opção certa, faça um `for` simulando a geração de dados (Ex: Gerando 3 relatórios, e imprimindo "Relatorio 1, Relatorio 2...").

Entrega: Código integrando um `for` aninhado dentro dos `cases` de um `switch`.

7. Desafio: Limpador de Array de Dados (Truthy/Falsy)

Descrição: Você recebeu da API a seguinte lista suja: ["João", "", null, "Maria", 0, "José"]. Crie um `for` que varra a lista. Faça um teste de conversão booleana na variável e passe os valores válidos para uma nova lista chamada `usuariosValidos`.

Entrega: Um array contendo apenas os dados genuínos retornados.

8. Desafio: Loop Seguro e Auditoria com IA

Descrição: Escreva um código intencionalmente quebrado contendo um Loop Infinito (ex: um `while(true)` sem `break`). Entregue esse código para uma IA (como Gemini ou ChatGPT) e peça para ela explicar *por que* seu navegador travaria.

Entrega: Um arquivo de texto documentando o código falho e o resumo da explicação que você obteve da IA, exercitando a depuração assistida de forma ética.

9. Desafio: Escape de Matriz (Uso de Labels)

Descrição: Baseado nas orientações do livro de David Flanagan, nós podemos usar *Labels* para saltar de laços múltiplos. Crie um laço dentro de outro (para percorrer uma tabela X,Y). Quando os laços acharem uma variável `corrompido = true`, dê um `break` na Label externa, parando tudo completamente.

Entrega: Algoritmo implementado com sintaxe de Rótulos (Label) validado.

10. Desafio: Fatiamento de Usuários do Sistema

Descrição: O sistema requisita que não sejam cadastrados nomes com menos de 3 caracteres. Você tem a lista: ["Oi", "Ana", "Lu", "Carlos", "Ze", "Edu"]. Crie um código que percorra a lista, extraia quem está irregular e pare o script imediatamente (usando `break`) se o sistema tentar avaliar o 4º erro, informando "Lote muito defeituoso. Suspenso".

Entrega: Lógica aplicada demonstrando domínio profundo sobre as condições de avanço e interrupção de fluxo.

Conexão com a Situação de Aprendizagem: Tudo o que fizemos hoje reflete a exigência real da nossa SA: o Dashboard interativo. Toda vez que um administrador do nosso projeto clicar em "Filtrar clientes ativos", não precisaremos ir até o servidor e sobrecarregá-lo recarregando a página (processo custoso). O JavaScript, no próprio computador do usuário, usará os laços `for` e as condicionais `switch` e `if` que treinamos hoje para varrer milhares de dados JSON em milissegundos, tomar as decisões com base na regra de negócio local, otimizando o sistema, economizando banda e gerando alta qualidade de usabilidade.

Próxima Aula: Na Aula 17 abordaremos os fundamentos das **Arrow Functions** e da **programação funcional**, focando na **manipulação** eficiente de **dados** através de **métodos** como **map**, **filter**, **reduce**, **spread** e **destructuring**. Aprenderemos sobre **escopo de variáveis** e a importância das **funções puras** para transformar informações sem gerar efeitos colaterais, vocês irão aplicar os conceitos em uma atividade prática, criando e validando utilitários de dados diretamente no console.